Exercise 2: Text-based Software Analysis

1 Regular Expressions

Consider the following code.

```
1 int order = atoi("10");
2 FILE *log = fopen("orders.log", "w");
3 sendMessage(7, "Order received");
4 if (order < 0) {
5     int status = system("printf 'Invalid order\\n'");
6     exit(status);
7 }
```

For each of the below software properties, do three things:

1. Provide a meaningful regular expression that matches the violations to the property **in the above code**.
2. Explain the strings this regular expression covers.
3. In the GitHub, Jupyter notebook `text-analysis.ipynb` contains code so that you can test your expressions on example program `policies.c`. Run your final solutions on `policies.c` and report, for each solution, the exact text your solution matches.

The properties:

a. φ_0 = No call to function exit

b. φ_1 = No call to function system.

c. φ_2 = No call to fopen uses truncating mode "w".

d. φ_3 = No call to the deprecated two-argument sendMessage form.

2 Formatting and False Positives

a. In the Jupyter notebook, `policies-formatted.c` has the same operations as `policies.c`, but different formatting. Predict which of your regular expressions still work, then test and try to improve them.

b. Inspect all matches your regular expressions produce for program `policies-benign.c`. Explain whether false positives appear and why.

3 For Enthusiasts: API Migration

If you finish the above tasks early:

a. Apply your regular expression for φ_3 = *No call to the deprecated two-argument sendMessage* to program `api-migration.c` and try to improve to be as precise as possible. You may write Python code for your analysis that is more complicated than a single regular expression. For example: count opening '(' and closing ')'

   This task is not expected to be solved perfectly.